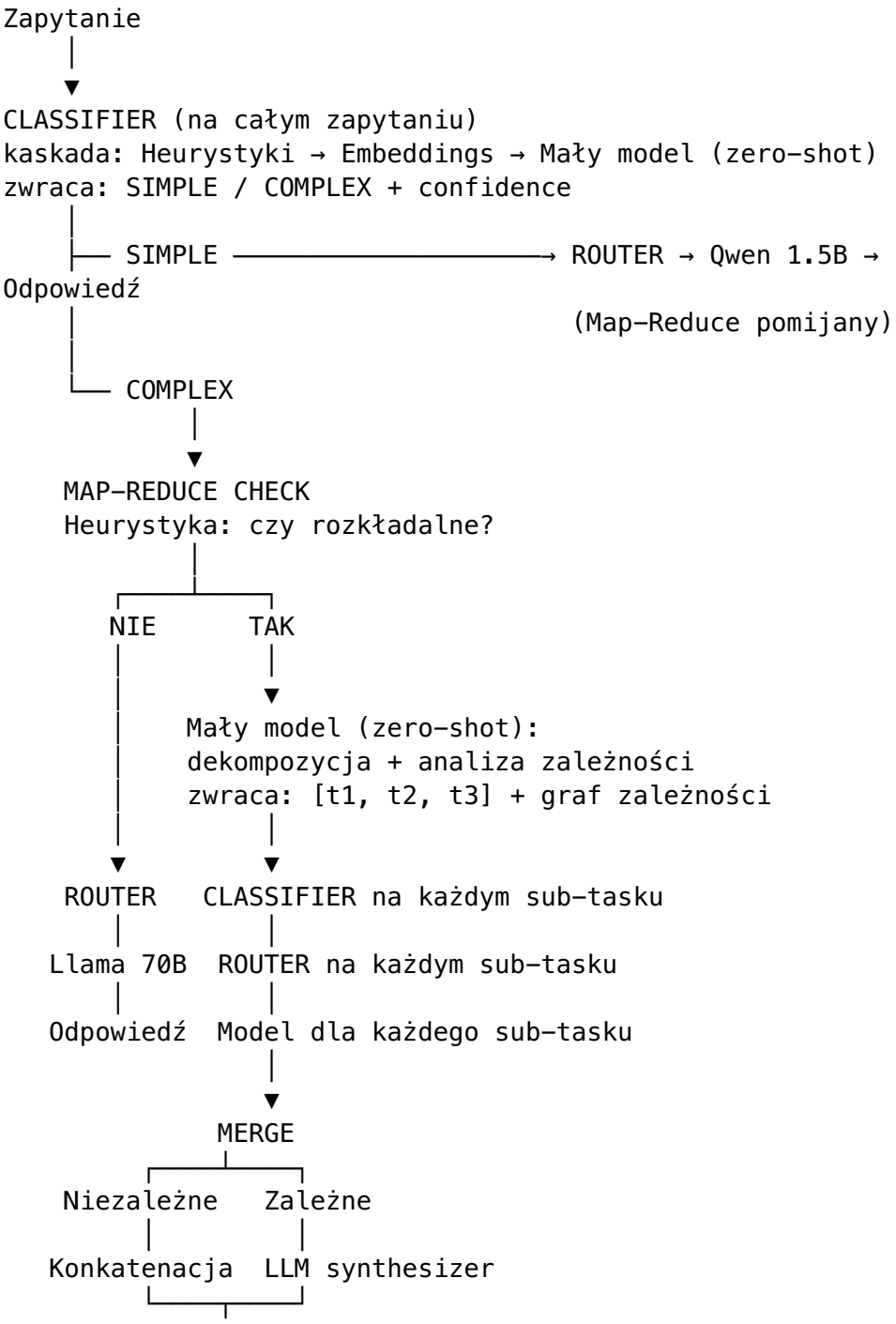
Dynamiczny i semantyczny routing zapytań — notatki projektowe

---

Pipeline — kolejność działania



```
        ▼
Odpowiedź finalna
```

---

1. CLASSIFIER

Moduł oceniający złożoność zapytania. Działa jako **pierwszy krok pipeline'u** — na całym zapytaniu. Wynik używany również do benchmarków i pomiaru jakości odpowiedzi.

Podział klas złożoności

Dwie klasy: **SIMPLE / COMPLEX** - Jednoznaczna granica — łatwiejsza do annotacji i kalibracji - Bezpośrednie porównanie z literaturą (RouteLLM, FrugalGPT) - Jeden próg decyzyjny zamiast dwóch

Metody oceny trudności — kaskada

Trzy warstwy działają kaskadowo — minimalizuje to narzut latencji.

```
Zapytanie / sub-task
        │
        ▼
[1] Heurystyki  (<1ms)
    │  pewny wynik → decyzja
    │  niepewny    ↓
[2] Embeddings  (~5–15ms)
    │  pewny wynik → decyzja
    │  niepewny    ↓
[3] Mały model zero-shot  (~10–20ms)
    └─ decyzja finalna
```

Warstwa 1 — Heurystyki

System punktowy (scoring) — każda cecha dodaje punkty, suma decyduje o klasie: - Długość promptu w tokenach - Obecność słów kluczowych: `analyze`, `compare`, `implement`, `step by step`, itp. - Liczba niezależnych poleceń / kroków w zapytaniu

Przykład:

```
score = 0
if token_count > 50:   score += 1
if token_count > 150:  score += 2
if token_count > 400:  score += 3
for kw in keywords:
    if kw in prompt: score += 2
if polecen >= 2:   score += 2
if polecen >= 4:   score += 3

# jeden próg:
```

```
if score >= THRESHOLD: → COMPLEX
else:                  → SIMPLE
```

"Pewność" heurystyki = odległość sumy punktów od progu. Jeśli score bliski progu → warstwa 2. Progi i wagi konfigurowane w YAML, kalibrowane empirycznie na validation set — próbujemy kolejne wartości progów, mierzymy wyniki, wybieramy najlepszą kombinację.

Warstwa 2 — Embeddings

- W bazie przechowywane są dwa centroidy — dla SIMPLE i COMPLEX
- Centroidy wyznaczane ze zbioru treningowego (validation set z etykietami)
- Nowe zapytanie wektoryzowane przez model embeddings z DeepFellow
- Pewność = różnica cosine similarity między centroidem COMPLEX a SIMPLE

```
similarity do centroidu COMPLEX = 0.87
similarity do centroidu SIMPLE  = 0.81
pewność = 0.87 - 0.81 = 0.06  → niska → warstwa 3

similarity do centroidu COMPLEX = 0.91
similarity do centroidu SIMPLE  = 0.55
pewność = 0.91 - 0.55 = 0.36  → wysoka → decyzja: COMPLEX
```

Warstwa 3 — Mały model (zero-shot)

Jeśli confidence embeddings niski — mały model (Qwen 1.5B) zapytany wprost:

```
Oceń złożoność zapytania. Odpowiedz: SIMPLE lub COMPLEX.
Zapytanie: "{prompt}"
```

Pewność = wartość softmax zwycięskiego tokenu. Koniec kaskady — zawsze zwraca decyzję.

---

2. ROUTER

Moduł wybierający model docelowy na podstawie wyniku klasyfikatora.

```python
def router(request, complexity_result):
    # na podstawie complexity_result.complexity i
        complexity_result.confidence
    # wybiera model z konfiguracji
    return model
```

Plik konfiguracyjny (YAML)

Jeden plik konfiguracyjny dla całego systemu: - Progi scoringu i wagi heurystyki (Warstwa 1) - Próg confidence embeddings (Warstwa 2) - Przypisanie klas do modeli (SIMPLE → Qwen 1.5B, COMPLEX → Llama 70B) - Fallback policy — gdy confidence klasyfikatora jest niski: przekieruj do Llama 70B

Progi kalibrowane empirycznie na validation set — bez zmiany kodu. Dla każdej kombinacji wartości progów puszczamy cały validation set przez classifier, mierzymy F1 i błędy A/B, wybieramy kombinację z najlepszym wynikiem.

Integracja z DeepFellow

Router działa jako Custom Endpoint wywoływany przed `/chat/completions` — nie modyfikuje core'u frameworka.

---

3. MAP-REDUCE

Mechanizm obsługi zapytań wieloetapowych. Odpala się tylko dla klasy COMPLEX.

Ocena rozkładalności — heurystyka wstępna

Szybki filtr przed wywołaniem modelu. Zapytanie jest kandydatem do dekompozycji gdy: - Zawiera spójniki enumeracyjne: *"i", "oraz", "a także", "po czym", "następnie"* - Ma strukturę wielopunktową (numerowaną lub bulletową) - Liczba niezależnych poleceń przekracza próg

Jeśli heurystyka nie wykryje wieloetapowości → zapytanie idzie do Routera. Jeśli wykryje kandydata → przekazuje do małego modelu.

Dekompozycja i analiza zależności — mały model (zero-shot)

Mały model jednym wywołaniem zwraca listę sub-tasków i graf zależności:

```
Rozłóż to zadanie na podzadania. Dla każdego określ:
- treść podzadania
- czy wymaga wyniku poprzedniego podzadania (tak/nie)
Zwróć JSON.
```

Każdy sub-task przechodzi przez Classifier i Router osobno — może trafić do innego modelu.

MERGE

Strategia scalania zależy od grafu zależności wykrytego przy dekompozycji.

Sub-taski niezależne → konkatenacja Odpowiedzi sklejane w ustalonej kolejności. Zero latencji, zero kosztu.

Sub-taski zależne → LLM synthesizer Odpowiedzi trafiają do modelu z promptem łączącym w jedną spójną całość. Dodatkowe wywołanie modelu = dodatkowa latencja, ale konieczne dla spójności.

---

4. DANE TESTOWE

Dane służą do **ewaluacji i kalibracji progów** — nie do trenowania żadnego modelu.

Każde zapytanie w zbiorze musi mieć: - treść zapytania - etykieta SIMPLE / COMPLEX - kategoria: `CODING`, `REASONING`, `CREATIVE_WRITING`, `TRANSLATION`, `FACTUAL_QA`, `MATH`

Kategorie potrzebne po to, żeby zobaczyć z jakimi typami zapytań classifier ma największe problemy.

Źródła danych i etykietowanie

Chatbot Arena

Publiczny zbiór realnych rozmów użytkowników z różnymi modelami. Użytkownicy głosowali który model odpowiedział lepiej. Etykietę SIMPLE / COMPLEX wnioskujemy z głosowań:

```
mocny model wygrał wyraźnie  →  COMPLEX
remis (oba odpowiedziały podobnie)  →  SIMPLE (słabszy model
wystarczył)
```

Wymagane filtrowanie: - wybieramy tylko bitwy gdzie różnica Elo między modelami jest duża (mocny vs słaby) - odrzucamy zapytania poniżej ~16 tokenów - sprawdzamy balans klas (żeby nie mieć 80% COMPLEX)

Podejście z remisem = SIMPLE jest zgodne z metodologią RouteLLM — można się powołać w pracy.

LLM-generated

Pytania generowane przez model LLM (inny niż Qwen 1.5B i Llama 70B — żeby nie testować na danych z tych samych modeli). Etykieta SIMPLE / COMPLEX nadawana automatycznie w prompcie generowania. Ważne żeby generować wszystkie trzy typy: wyraźnie SIMPLE, wyraźnie COMPLEX oraz przypadki graniczne (na granicy między klasami — gdzie classifier może się mylić).

Ręczna praca: przegląd ~15–20% próbki czy model nie halucynuje etykiet.

Ważne: model generujący dane musi być inny niż modele testowane w systemie.

Podział zbioru

```
Cały zbiór (100%)
    │
    ├── 70% — validation set
    │         do kalibracji progów w YAML
    │         można eksperymentować dowolnie
    │
    └── 30% — test set
```

```
                zamknięty podczas całej kalibracji
                otwierany dokładnie raz — finalny pomiar do pracy
```

Test set zamknięty żeby nie "wyciec" — gdybyśmy kalibrowali progi patrząc na test set, wyniki byłyby zawyżone i nierzetelne.

Kalibracja progów

Dla każdej kombinacji wartości progów w YAML puszczamy cały validation set przez classifier, porównujemy wyniki z etykietami i mierzymy metryki. Wybieramy kombinację z najlepszym wynikiem. Powtarzamy aż wyniki satysfakcjonujące. Dopiero wtedy otwieramy test set — jeden raz, finalny pomiar.

```
przykład dla jednego progu THRESHOLD:

THRESHOLD=3 → F1=0.71, błąd A=12%
THRESHOLD=5 → F1=0.81, błąd A=4%   ← wybieramy
THRESHOLD=7 → F1=0.74, błąd B=25%
```

Metryki ewaluacji klasyfikatora

Accuracy + F1 score — F1 liczony względem klasy COMPLEX, bo błędne przypisanie COMPLEX → SIMPLE jest najkosztowniejsze jakościowo.

Błędy routingu:

```
Błąd A (critical) — zapytanie COMPLEX → Qwen 1.5B (za słaby)
                    konsekwencja: niska jakość odpowiedzi

Błąd B (costly)   — zapytanie SIMPLE → Llama 70B (za duży)
                    konsekwencja: zmarnowane zasoby GPU
```

Miara A/B pozwala świadomie przesuwać próg — np. akceptuj więcej błędów B żeby wyeliminować błędy A.

Miara jakości odpowiedzi (Quality Degradation)

Mierzona tylko dla zapytań gdzie router wybrał Qwen 1.5B (mniejszy model).

```
To samo zapytanie puszczane dwa razy:

Statyczna:  zapytanie → Llama 70B        → odpowiedź_referencyjna
Router:     zapytanie → Classifier → Router
                      → Qwen 1.5B        → odpowiedź_routera

Porównanie: odpowiedź_referencyjna  vs  odpowiedź_routera
```

Dla Map-Reduce: porównujemy zmergowaną odpowiedź routera z odpowiedzią Llama 70B na oryginalne zapytanie (bez dekompozycji). Jako miara dodatkowa: jakość każdego sub-tasku osobno.

ROUGE — algorytmiczne porównanie tekstu: - Liczy wspólne n-gramy między odpowiedzią routera a referencyjną - Problem: różnice syntaktyczne przy semantycznie poprawnej odpowiedzi zaniżają wynik - Nadaje się do: tłumaczeń, streszczeń, formatowania

GPT-as-Judge — duży model ocenia jakość odpowiedzi w skali 1–5: - Wolniejszy i kosztowniejszy - Dobrze radzi sobie z kodowaniem i złożonym rozumowaniem - Unika problemu syntaktycznych różnic przy ocenie sensu

Strategia: ROUGE dla klasy SIMPLE, GPT-as-Judge dla klasy COMPLEX.

---

5. BENCHMARKI — Analiza zysków

Porównanie **architektura statyczna** (zawsze Llama 70B) vs. **architektura z routerem**.

Metryka	Opis
Latency	Średni czas odpowiedzi na zapytanie
GPU utilization	Zużycie GPU w czasie pod obciążeniem
Throughput	Liczba obsłużonych zapytań na sekundę
Response Quality	Wynik ROUGE / GPT-as-Judge

Wyniki raportowane w metrykach PGR/APGR/CPT zgodnych z RouteLLM — umożliwia bezpośrednie porównanie z literaturą.

Oczekiwany wynik: architektura z routerem osiąga niższą latencję i wyższy throughput przy akceptowalnej degradacji jakości odpowiedzi.

---

6. METRYKI PGR / APGR / CPT

Metryki z RouteLLM — umożliwiają bezpośrednie porównanie wyników z literaturą.

PGR (Performance Gap Recovered)

Ile procent różnicy jakości między słabym a mocnym modelem router odzyskuje.

```
PGR = (jakość_routera - jakość_słabego) / (jakość_mocnego - jakość_słabego)

Przykład:
jakość Qwen 1.5B  = 6.5/10
jakość Llama 70B  = 9.0/10
jakość routera    = 8.2/10

PGR = (8.2 - 6.5) / (9.0 - 6.5) = 68%
```

PGR=0 oznacza jakość Qwen 1.5B, PGR=1 oznacza jakość Llama 70B.

APGR (Average PGR)

Jeden numer podsumowujący router niezależnie od wybranego progu. Liczone jako pole pod krzywą PGR dla wszystkich możliwych wartości progu — od 0% do 100% zapytań kierowanych do mocnego modelu.

```
losowy routing = APGR 0.5
dobry router   = APGR 0.8+
```

CPT (Call-Performance Threshold)

Ile minimum procent zapytań musi trafić do mocnego modelu żeby osiągnąć PGR = X%.

```
CPT(50%) = 13% oznacza:
wystarczy wysłać tylko 13% zapytań do Llamy 70B
żeby odzyskać 50% jej przewagi jakościowej nad Qwen 1.5B
```

Im niższy CPT tym lepszy router — osiąga dobrą jakość przy małym udziale mocnego modelu.